

Kubernetes Beginner Guide

Chapter 4: Kubernetes Storage

Explained in simple English, figure by figure. Theory only, no code.

What you will learn

- Why data written inside a Pod is lost when the Pod is replaced
- The three main **Volume** types: emptyDir, hostPath, and cloud or network disks
- What a **PersistentVolume (PV)** and a **PersistentVolumeClaim (PVC)** are, and how they bind
- What a **StorageClass** is and how **dynamic provisioning** creates disks automatically
- Why a **StatefulSet** gives each Pod its own stable, permanent storage
- A full practical example: a database that survives a Pod crash

How to read this chapter

Every topic has one figure. Look at the figure first, then read the explanation under it. Each explanation walks through every part of the picture, gives the reason it exists, and ends with a real-life example. There are six figures in total.

1. Why containers need persistent storage

In Chapter 2 you learned that Pods are temporary, like paper cups. A crash, an update, or a node failure can delete a Pod at any time. This becomes a serious problem the moment your app needs to **remember** anything, such as a database, uploaded files, or a shopping cart.

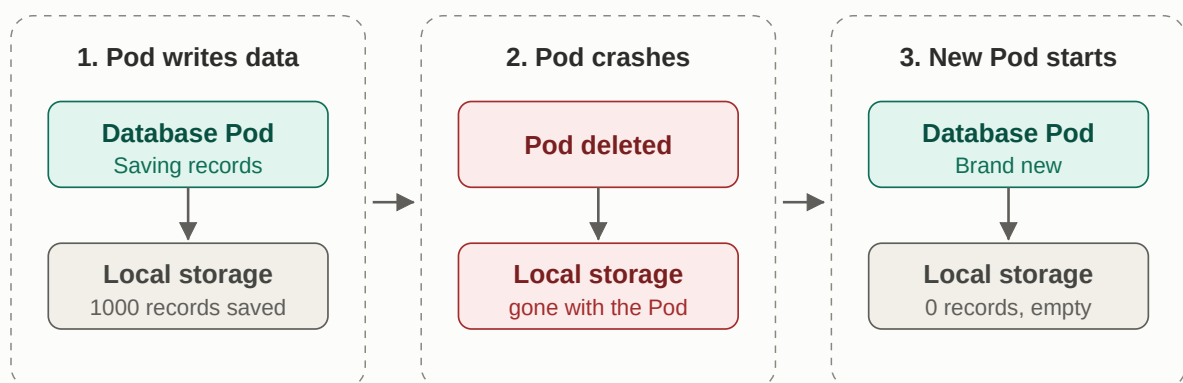


Figure 1. Data saved only inside a Pod is lost forever when that Pod is replaced.

Explaining Figure 1

Panel 1: The Pod writes data. A database Pod saves 1000 customer records into storage that lives inside the Pod itself, a normal filesystem, exactly like a folder on a laptop.

Panel 2: The Pod crashes. By default, whatever was written inside that Pod is tied to that Pod's own lifetime. A crash, a node failure, or even a routine update can delete a Pod.

Panel 3: A new Pod starts, with nothing. The ReplicaSet creates a replacement Pod, but it is brand new, with brand new, empty local storage. All 1000 records are gone. The app "works" again, but the data does not.

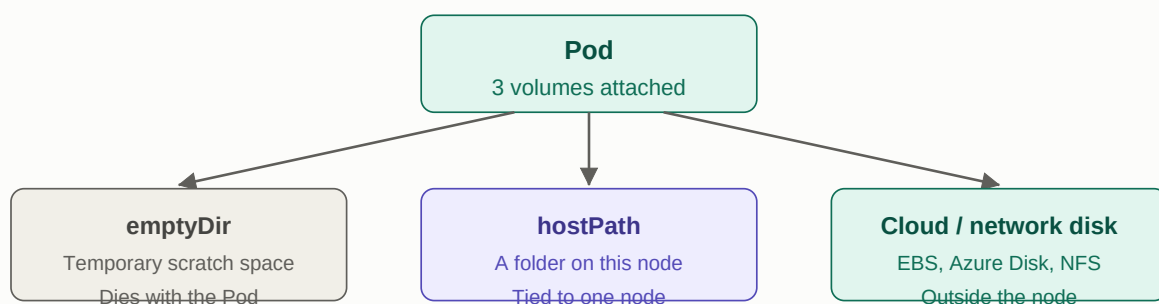
The fix. Kubernetes lets you attach storage that lives **outside** the Pod's own lifetime. The Pod uses it while it runs, but if the Pod is deleted and replaced, the new Pod can attach to the very same storage and find all the old data still there. That external storage is what the rest of this chapter is about.

Real-life example

Writing notes on a whiteboard in a hotel room you are about to check out of, versus writing them in a notebook you carry with you. The whiteboard (local storage) is wiped clean for the next guest. The notebook (persistent storage) is yours no matter which room you get next.

2. Volumes and Volume types

A **Volume** is a general word for "a storage location that a Pod can use." It is attached to the Pod, and one or more containers inside can read and write to it. There are many volume types; here are the three groups to know as a fresher.



Only the outside disk survives if the Pod is deleted or the node fails.

Figure 2. Only storage that lives outside the node survives a Pod or node failure.

Explaining Figure 2

emptyDir (gray, left). An empty folder created the moment the Pod starts. Good for temporary scratch space, a cache, or sharing files between two containers in the same Pod. It is deleted forever when the Pod is deleted.

hostPath (purple, middle). This points to a folder that already exists on the **node's own disk**. Unlike emptyDir, this data survives if a new Pod is created on the **same node**. But if the new Pod lands on a different node, which the Scheduler decides, it will not see this data at all. This makes hostPath risky, and it is mostly used for testing or special system Pods.

Cloud or network disk (teal, right). Real, independent storage living **outside every node**: an AWS EBS volume, an Azure Disk, a Google Persistent Disk, or a network filesystem like NFS. It survives a

Pod being deleted, and it even survives if the whole node fails, since a new Pod on a different node can still attach to it.

The important takeaway. Only the third type gives true persistence in a multi-node cluster. This is exactly what PV and PVC are built to manage.

Real-life example

emptyDir is a sticky note you throw away when you leave the desk. hostPath is a locked drawer in that specific desk, useless if you get moved to a different desk tomorrow. The cloud disk is a safety deposit box at a bank branch that any employee, at any desk, in any city, can access with the right key.

3. PersistentVolume (PV) and PersistentVolumeClaim (PVC)

This works like a rental agreement, and it is one of the most asked concepts for freshers.

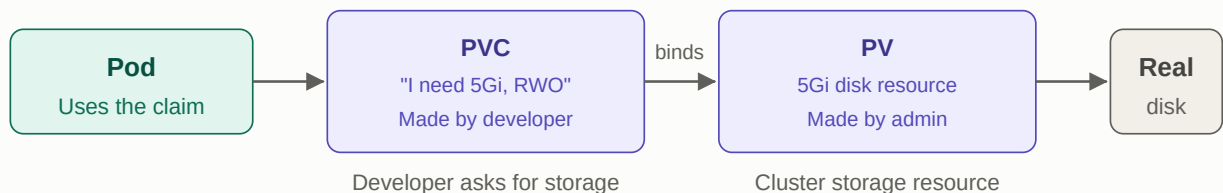


Figure 3. A PVC is a request for storage, which binds to a matching PV.

Explaining Figure 3

PersistentVolume (PV), right. The actual piece of storage available in the cluster, for example a real 5Gi cloud disk. It is a **cluster-level resource**, not owned by any Namespace or Pod. Usually created by a cluster **admin**, pointing to the real underlying disk.

PersistentVolumeClaim (PVC), middle. A **request** made by a developer: "I need 5Gi of storage, readable and writable from one Pod at a time." It does not create storage, only asks for it.

Binding. Kubernetes finds a PV that matches what the PVC asks for, size, access mode, and so on, and binds them one to one. No other PVC can then use that same PV.

Pod, left. The Pod never talks to the PV directly. It only mentions the PVC by name. The developer only says what they need, and never needs to know which exact disk they got.

Why split PV and PVC into two objects?

Who	What they care about
Cluster admin	Where the real disks come from, their cost, backup policy
Developer	"I just need 5Gi of storage for my app," nothing more

What survives what? Deleting the Pod does **not** delete the PV or its data. Even deleting the PVC does not always delete the data, it depends on the reclaim policy: **Retain** keeps the data for an admin to

handle, and **Delete** removes the disk automatically.

Real-life example

A PV is a storage locker already built into an office building, owned by the building. A PVC is the rental slip an employee fills out: "I need one locker, at least 5Gi." They never need to know its exact location.

4. StorageClass and dynamic provisioning

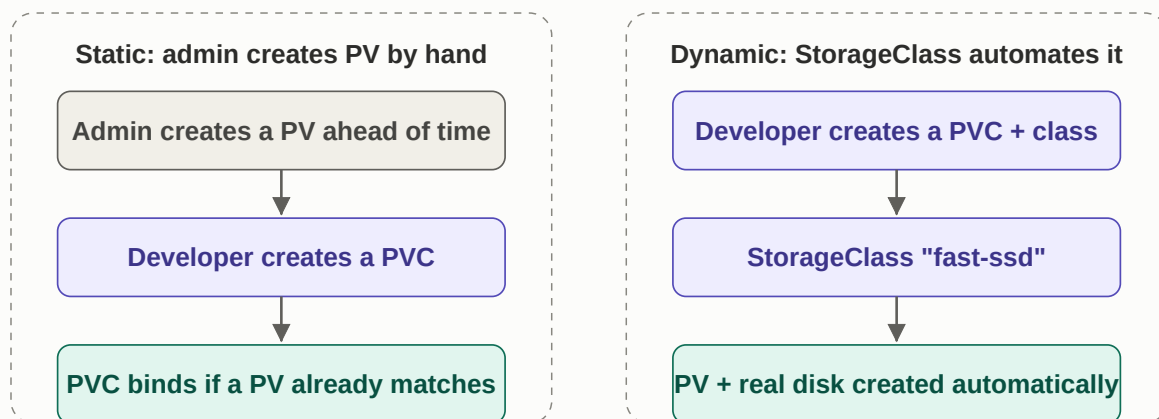


Figure 4. A StorageClass lets Kubernetes create matching PVs automatically.

Explaining Figure 4

Left: Static provisioning (manual). This is exactly what Figure 3 showed. An admin must create PVs ahead of time, guessing how many developers will need storage. If a PVC does not match any existing PV, it stays unbound forever, and the Pod cannot start.

Right: Dynamic provisioning (automatic). The developer's PVC names a **StorageClass**, for example fast-ssd. Kubernetes then automatically talks to the storage system, creates a brand new real disk, creates a matching PV, and binds it to the PVC. All within seconds, with no admin needing to act.

What is a StorageClass, exactly? A template describing **how** to create storage automatically: which storage system to use, disk speed or type, and reclaim policy. A cluster can have several, for example fast-ssd for databases and standard-hdd for backups, and a developer simply picks one by name.

This is how real clusters work. In almost every company cluster, developers never touch a PV directly. Minikube and kind even come with a default StorageClass already installed, so often a PVC alone, with no storageClassName, still works.

Real-life example

Static provisioning is calling the building manager and asking them to personally build a new storage locker, then waiting. Dynamic provisioning is a vending machine: select "medium locker," and it is built and handed to you automatically, no manager involved.

5. StatefulSet and storage

A Deployment (Chapter 2) treats all its Pods as identical and interchangeable, with random names. That is perfect for a stateless web server, but a **database cluster** usually needs a **stable name** per replica, and each replica needs to keep its **own separate storage**, even after being restarted or rescheduled. This is what a **StatefulSet** is built for.

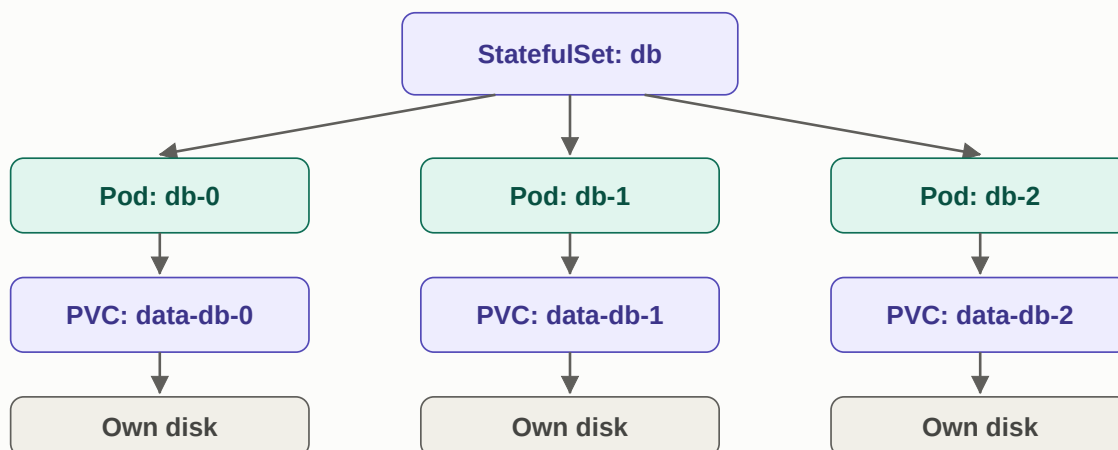


Figure 5. Each Pod in a StatefulSet keeps its own name and its own dedicated storage forever.

Explaining Figure 5

StatefulSet, top. Like a Deployment, you set how many replicas you want, here 3. But it manages Pods very differently.

Stable, ordered Pod names. Instead of random names, a StatefulSet creates Pods named db-0, db-1, db-2, always in that order, one at a time. If db-1 is deleted and recreated, its replacement is still called db-1, never a new random name.

One PVC per Pod, paired for life. Each Pod gets its **own** PVC matched to its name: db-0 always uses data-db-0, and so on. If db-1 crashes and is replaced, the new Pod reconnects to the very same data-db-1 claim, and therefore the same disk with the same data. Compare this with a Deployment, where all Pods share an identical PVC or have no storage identity at all.

Why does a database need this? Real database clusters, such as MySQL replicas or MongoDB shards, are not interchangeable. Each node often holds a different part of the data, or has a role such as "primary" or "replica." A StatefulSet guarantees this pairing never breaks.

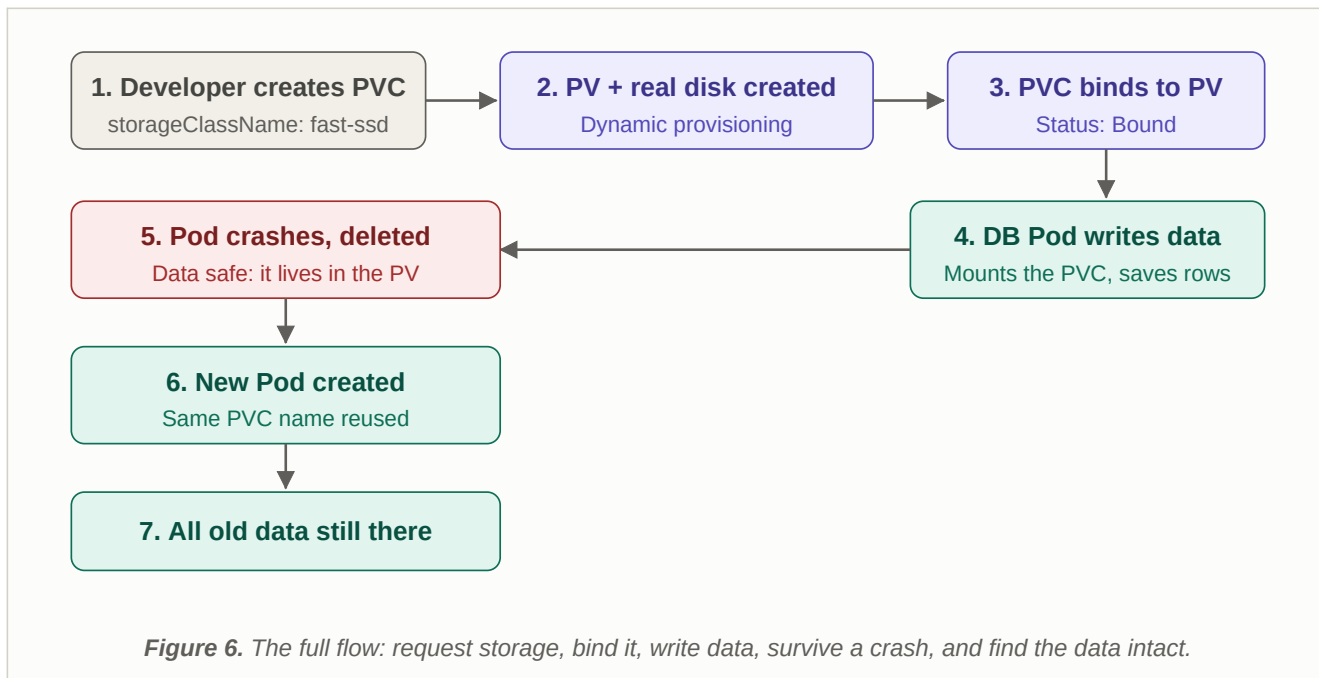
StatefulSet vs Deployment, in one line: use a Deployment for stateless apps where every Pod is identical and disposable, and use a StatefulSet for apps where each Pod needs a stable identity and its own permanent storage.

Real-life example

A Deployment is like a taxi rank: any driver can serve any customer. A StatefulSet is like assigned school lockers: locker 0 always belongs to student 0, even if the locker is repainted, it is given back to that same student with their belongings inside.

6. Practical database storage example, start to finish

This ties together every idea from this chapter into one real journey, solving the exact problem shown in Figure 1.



Explaining Figure 6

- Developer creates a PVC**, asking for storage and naming a StorageClass such as fast-ssd. No thinking about disks, only "how much space do I need."
- A PV and a real disk are created automatically**, through dynamic provisioning (Figure 4). No admin action needed.
- The PVC binds to the PV**. Status shows Bound, meaning the request and the real storage are officially paired.
- The database Pod starts and mounts the PVC** as a folder inside itself, then writes its data there, except this time the data lives in the PV, not inside the Pod.
- The Pod crashes and is deleted**, just like Figure 1's Panel 2. This time, the data is untouched, because it was never inside the Pod.
- A new Pod is created** by the ReplicaSet or StatefulSet, configured to use the **same PVC name** as before.
- All the old data is still there**. The new Pod mounts the same PVC, finds the same PV, finds the same disk, and reads every record written before the crash.

This is the complete answer to Figure 1's problem. Local storage disappears with the Pod. Storage attached through a PVC does not, because the PVC and PV exist independently of any single Pod's lifetime.

One more detail for real databases. If this is a multi-replica database using a StatefulSet (Figure 5), each replica, db-0, db-1, db-2, repeats this exact same flow independently, each with its own PVC, always coming back to its own specific data after any crash.

Chapter 4 summary

Figure	Main idea
1	A Pod's own local storage is deleted along with the Pod, so anything important is lost on crash or replacement
2	emptyDir is temporary, hostPath is tied to one node, only outside cloud or network storage truly persists
3	A PVC is a developer's request for storage, and it binds to a matching PV, the real storage resource
4	A StorageClass lets Kubernetes create PVs automatically (dynamic provisioning), instead of an admin by hand
5	A StatefulSet gives each Pod a stable name and its own permanent PVC, ideal for databases
6	PVC requests storage, a PV is created and bound, the Pod writes data, and after a crash the new Pod finds the same data

Check yourself

Try to answer these in your own words, without looking at the notes:

- 1 Why is data lost when a Pod using only local storage is deleted and replaced?
- 2 What is the difference between emptyDir, hostPath, and a cloud or network disk?
- 3 What is the difference between a PV and a PVC? Which one does the developer create?
- 4 What problem does a StorageClass solve compared to an admin creating PVs by hand?
- 5 Why does a database usually need a StatefulSet instead of a Deployment?
- 6 In Figure 6, why does the new Pod in step 6 still see the old data from step 4?

Next: Chapter 5, Configuration and Secrets

You will learn how **ConfigMaps** and **Secrets** feed settings and sensitive information into Pods, without baking them into the container image.